

Ejercicio 1

Los árboles van a pedirse
recursivos

```
public int numDatos() {
```

```
    return numDatosRec(raiz);
```

```
}
```

```
private int numDatosRec(NodoArbol nodo) {
```

```
    int res = 0;
```

```
    if (nodo != null) {
```

```
        res = numDatosRec(nodo.getIzquierdo()) + numDatosRec(nodo.getDerecho()) + 1;
```

```
    }
```

```
    return res;
```

```
}
```

```
public boolean contieneEnHoja(int dato) {
```

Caso base, árbol vacío, se
encuentra hoja.

```
    return contieneEnHoja(actual, dato);
```

```
}
```

```
private boolean contieneEnHoja(NodoArbol actual, int dato) {
```

```
    boolean res;
```

```
    if (actual == null) {
```

```
        res = false; ← caso base árbol vacío
```

actual es una hoja → } else if (actual.getIzquierdo() == null && actual.getDerecho() == null) {

```
    res = (actual.getData() == dato);
```

actual empieza en una rama o raíz → } else {

```
    res = this.contieneEnHoja(actual.getIzq(), dato);
```

```
    if (res == false) {
```

```
        res = this.contieneEnHoja(actual.getDer(), dato);
```

```
    }
```

```
    }
```

```
    return res;
```

```
}
```

```
public boolean equals (ArbolBinario a) {
```

```
    return this;
```

Solo en este caso
no hace falta getter, setter
porque son del mismo tipo
(ArbolBinario)

```
}
```

```
private boolean equals (NodoArbol actual1, NodoArbol actual2) {  
    boolean res;
```

```
    if (actual1 == null && actual2 == null) {
```

```
        res = true;
```

```
    } else if ((actual1 == null && actual2 != null) || (actual1 != null  
        && actual2 == null)) {
```

```
        res = false;
```

Recursivo → { else {

```
        res = (actual1.getData() == actual2.getData()) &&
```

```
            this.equals(actual1.getIzquierdo(), actual2.getIzq());
```

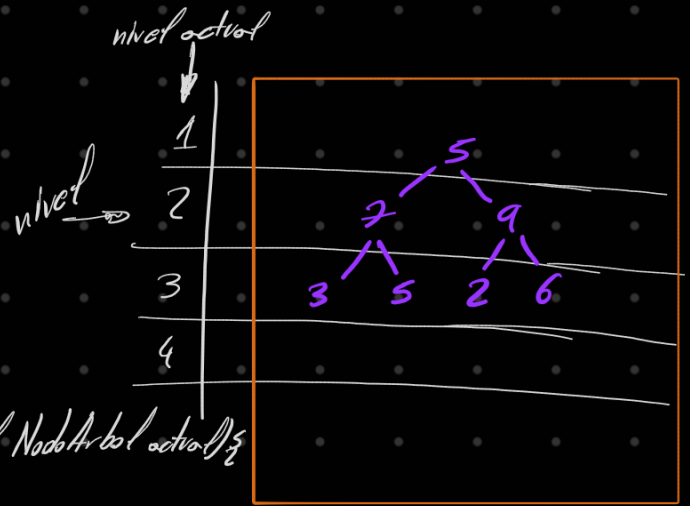
```
            && this.equals(actual1.getDer(), actual2.getDer());
```

```
    }
```

```

public int sumaDatosNivel(int nivel) {
    NodoArbol actual = raíz;
    return sumaDatosNivel(nivel, 1, actual);
}

```



Trabajamos con árboles binarios

```

private int sumaDatosNivel(int nivel, int nivelActual, NodoArbol actual) {
    int suma = 0;
    if (actual != null) {
        if (nivel == nivelActual) {
            suma = actual.getDato();
        } else {
            nivelActual++;
            suma = sumaDatosNivel(nivel, nivelActual, actual.getIzq()) +
                sumaDatosNivel(nivel, nivelActual, actual.getDer());
        }
    }
    return suma;
}

```

```

public int sumaDatosImparesDosHijos() {
    NodoArbol actual = raíz;
    return sumaDatosImparesDosHijosRec(actual);
}

```

```

private int sumaDatosImparesDosHijosRec(NodoArbol actual) {
    int suma = 0;
    if (actual == null) {
        suma = 0;
    } else {
        if ((actual.getDato() % 2) != 0 && actual.getIzq() != null &&
            actual.getDer() != null) {
            suma = actual.getDato();
        }

        suma = sumaDatosImparesDosHijosRec(actual.getIzq()) +
            sumaDatosImparesDosHijosRec(actual.getDer());
    }
    return suma;
}

```



```
public void invertir() {
```

```
}
```

```
private void invertirRec(NodoArbol actual) {
```

```
if (actual != null) {
```

```
    izq = actual.getDerecho();
```

```
    der = actual.getIzquierdo();
```

```
    actual.setDerecho(izq);
```

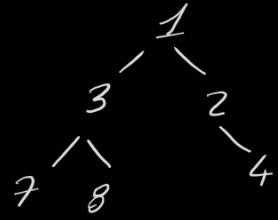
```
    actual.setIzquierdo(der);
```

```
    invertirRec(actual.getIzquierdo());
```

```
    invertirRec(actual.getDerecho());
```

```
}
```

```
}
```



profundidad máxima

b) public int altura() {

}

private int alturaRec(NodoArbol actual) {

int prof = 0;

if (actual != null) {

prof = 1 + alturaRec(actual.getIzquierdo());

int profder = 1 + alturaRec(actual.getDerecho());

if (prof < profder) {

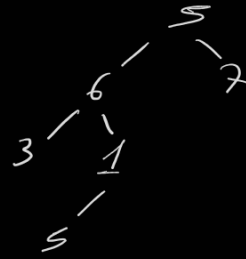
prof = profder;

}

}

return prof;

}



```
1) public boolean esUnivariado() {
```

```
    return esUnivariadoRec(raiz, raiz.getDato());
```

```
}
```

```
private boolean esUnivariadoRec(NodoArbol actual, int dato) {  
    boolean res = true;
```

```
    if (actual != null) {
```

```
        if (dato != actual.getDato()) {
```

```
            res = false;
```

```
        }
```

```
        boolean izq = esUnivariadoRec(actual.getIzquierdo(), dato);
```

```
        boolean der = esUnivariadoRec(actual.getDerecho(), dato);
```

```
        if (!izq || !der) { // Si izq = false o der = false
```

```
            res = false;
```

```
        }
```

```
    }
```

```
    return res;
```

```
}
```



Esto viene de sus hijos si estos dicen false sobrescriben lo del padre si dicen true, lo del padre se mantiene.

```

1) public boolean iguales(Arbol arbol) {

```

```

    return igualesRec(raiz, this.raiz);

```

```

}

```

```

private boolean igualesRec(NodoArbol 1, NodoArbol 2) {
    boolean res = false;

```

```

    if (1 == null && 2 == null) {
        res = true;

```

```

    } else if (1 != null && 2 != null) {

```

```

        if (1.getDato() == 2.getDato()) {

```

```

            boolean izq = igualesRec(1.getIzquierdo(), 2.getIzquierdo());

```

```

            boolean der = igualesRec(1.getDerecho(), 2.getDerecho());

```

```

            if (izq && der) {
                res = true;

```

```

            }

```

```

        }

```

```

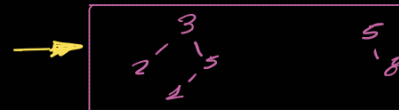
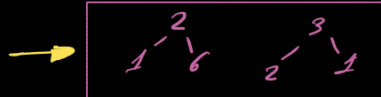
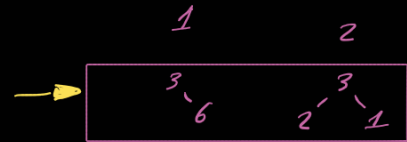
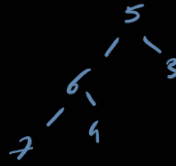
    }
    return res;

```

```

}

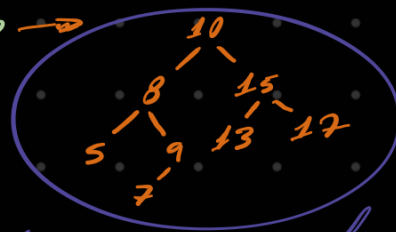
```



TAD diccionario en árbol binario ordenado →

(clave, dato)

↑ int, obj, ...

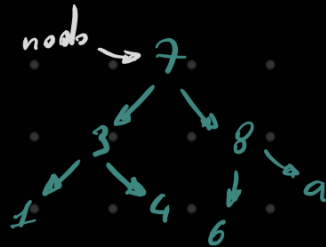


Los números son las claves

Array de claves en array en orden

int[] clave = {10, 8, 15, 5, 9, 13, 17, 7}

Metodo insertar en un árbol binario con diccionario



insert(2)

Se recorre el árbol comparando la clave yendo por der o izq hasta encontrar un hueco (un null).

Al encontrarlo se añade uniéndolo al padre y se tiene en cuenta el actualizar para el resto de padres y el count

Hay que poder actualizar el nodo de los padres así como el count.

En el caso de eliminar hay tres casos: ➡ Página 60 Tema 5 Árboles

Ejercicio 2 (Diccionario - TreeMap)

TreeMap < K, V > De Java. Util
↑ tipo del dato
↑ tipo de la clave

```
public static boolean sonAnagramas (String cadena1, String cadena2) {  
    TreeMap < Character, Integer> tm = new TreeMap <> ();  
    boolean res;  
    if (cadena1.length() == cadena2.length()) {  
        res = true;  
    } else {  
        res = false;  
    }  
  
    for (int i = 0; i < cadena1.length(); i++) {  
        char a = cadena1.charAt(i);  
        tm.put(a, i+1);  
    }  
  
    for (int j = 0; j < cadena2.length(); j++) {  
        char b = cadena2.charAt(j);  
        if (!tm.containsKey(b)) {  
            res = false;  
        }  
    }  
  
    return res;  
}
```

// Claves como caracteres, letras en orden alfabético, solo metemos un string en la estructura árbol y buscamos la segunda cadena en él.

Ejercicio 3 Usamos como clave el string y el dato un contador. Vamos guardando el contador y el string asociado.

Se recorre una vez, si no existe se crea y si existe se aumenta el dato.

```
public static String palabraMasRepetida(String[] palabras) {  
    TreeMap<String, Integer> tm = new TreeMap<>();  
    String masRepetida = palabras[0];  
    for (int i = 0; i < palabras.length; i++) {  
        if (!tm.containsKey(palabras[i])) {  
            tm.put(i, tm.get(i) + 1);  
            if (tm.get(masRepetida) < tm.get(i)) {  
                masRepetida = palabras[i];  
            }  
        } else {  
            tm.put(i, 1);  
        }  
    }  
    return masRepetida;  
}
```

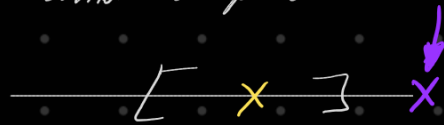
Ejercicios de conjuntos

4.-

Busqueda en profundidad recursiva

```
public int antecesorosComunes(int c1, int c2) {
    if (c1 <= c2) {
        return antecesorosRec(raiz, c1, c2);
    } else {
        return antecesorosRec(raiz, c2, c1);
    }
}
```

Coje el mayor de $c1$ - $c2$ y lo usa como intervalo para ver si va hacia derecha o izquierda y suma uno por ser antecesor común.



```
private int antecesorosComunesRec(NodoArbol nodo, int c1, int c2) {
    int res = 0;
```

```
    if (nodo != null) {
        if (nodo.get(clave) < c1) {
```

Damos por hecho $c1 < c2$

ultimo antecesor que se enaventre antes de $c2$

```
        res = 1 + antecesorRec(nodo.getDerecho(), c1, c2);
```

```
    } else if (nodo.get(clave) > c2) {
```

```
        res = 1 + antecesorRec(nodo.getIzquierdo(), c1, c2);
```

```
    } else if (nodo.get(clave) > 1 && nodo.get(clave) < 2) {
```

```
        res = 1; // > c1 && < c2
```

```
    }
```

```
    return res;
```

```
}
```

Se queda con $res=0$ cuando $= c1 || = c2$

